CASE STUDY 01

University Scholarship Allocation

1. PROBLEM STATEMENT

Design a program that checks scholarship applicants using aggregate marks, subject-wise minimum marks, attendance,
and annual family income. Eligible students are ranked, and ties are resolved by fixed rules.

2. OBJECTIVES

• Check every scholarship condition before accepting an applicant.

• Rank eligible applicants by aggregate marks.

• Resolve equal aggregate marks using subject marks, attendance, and income.

3. ASSUMPTIONS

• Aggregate marks and subject marks are percentages from 0 to 100.

• Eligibility requires aggregate >= 75, Mathematics >= 60, Physics >= 50, Computer Science >= 60, attendance >= 85, and

• annual income <= ■250,000.

• Tie order: higher aggregate, higher Mathematics, higher Physics, higher Computer Science, higher attendance, then

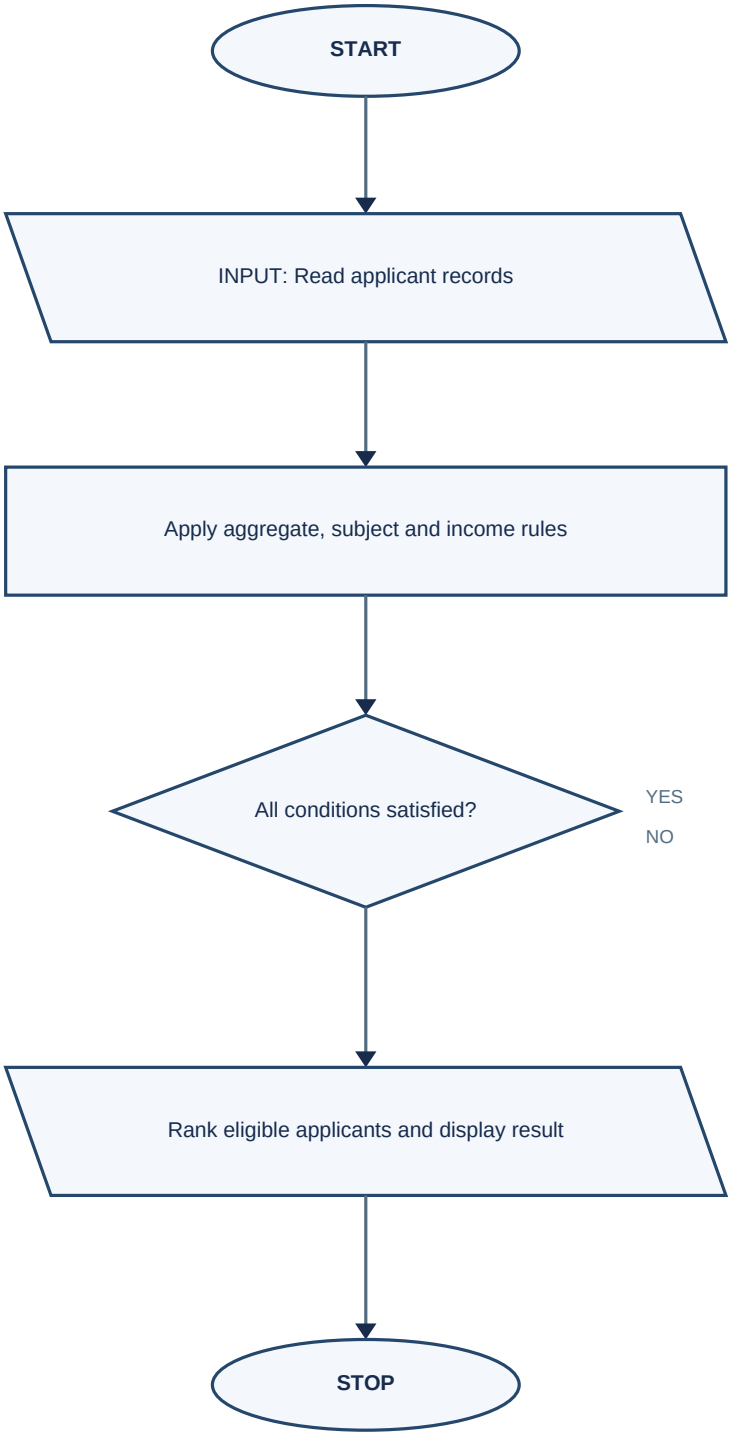
• lower income.

4. ALGORITHM

1. Read all applicant records.
2. Create eligible and not-eligible lists.
3. Check aggregate, each subject mark, attendance and income for every applicant.
4. Place applicants who satisfy every rule in the eligible list; place the others in the not-eligible list.
5. Sort eligible applicants by the stated ranking rules and display both lists.

5. FLOWCHART

Oval: START/STOP | Parallelogram: input/output | Rectangle: process/calculation | Diamond: decision



6. PSEUDOCODE

```
BEGIN
INPUT applicant records
eligible = empty list
not_eligible = empty list
FOR each applicant
CHECK aggregate, subject marks, attendance and income
IF every condition is satisfied
ADD applicant to eligible
ELSE
ADD applicant name to not_eligible
SORT eligible by aggregate, subject marks, attendance,
then lower income
DISPLAY ranked eligible list and not_eligible
END
```

7. PYTHON SOURCE CODE

```
applicants = [
["Asha", 92, 85, 80, 90, 95, 150000],
["Bharat", 88, 75, 70, 80, 90, 200000],
["Charan", 95, 45, 80, 90, 98, 100000],
["Divya", 88, 80, 75, 85, 95, 120000],
["Esha", 78, 70, 65, 75, 82, 100000]
]
eligible = []
not_eligible = []
for s in applicants:
name, agg, math, physics, cs, att, income = s
if (agg >= 75 and math >= 60 and physics >= 50
and cs >= 60 and att >= 85 and income <= 250000):
eligible.append(s)
else:
not_eligible.append(name)
eligible.sort(key=lambda s: (-s[1], -s[2], -s[3],
-s[4], -s[5], s[6]))
print("SCHOLARSHIP RANK LIST")
for rank, s in enumerate(eligible, 1):
print(rank, s[0], "- Aggregate:", s[1], "%")
print("Not eligible:", not_eligible)
```

8. TEST CASES & EXPECTED RESULTS

TEST INPUT / CONDITION REASONING EXPECTED RESULT

Asha 92%, all subject marks pass, 95% attendance, Eligible; Rank 1

income ■150000

Charan Mathematics 45% Not eligible: subject minimum not met

Divya and Bharat Both aggregate 88%; attendance 95% and 90% Divya ranks higher

9. SAMPLE OUTPUT

```
SCHOLARSHIP RANK LIST
1 Asha - Aggregate: 92 %
2 Divya - Aggregate: 88 %
3 Bharat - Aggregate: 88 %
Not eligible: ['Charan', 'Esha']
```

10. CONCLUSION

The program checks all scholarship rules, excludes applicants who fail any condition, and produces a transparent ranking

with deterministic tie-breaking.

SYLLABUS CONCEPTS Variables • Data Types • Operators • Conditions • Loops • Lists • Nested Lists • Tuples • String Operations